

Cat vs Dog Image Classifier

Project Analysis, Goals, and Theory

1 The Problem

The objective is to construct a binary image classifier:

$$f(\text{image}) \longrightarrow \{\text{cat}, \text{dog}\}.$$

More precisely, given an image x , the model estimates

$$P(\text{cat} \mid x), \quad P(\text{dog} \mid x).$$

For example, it may output $P(\text{cat} \mid x) = 0.07$ and $P(\text{dog} \mid x) = 0.93$, after which the application selects the class with the larger score.

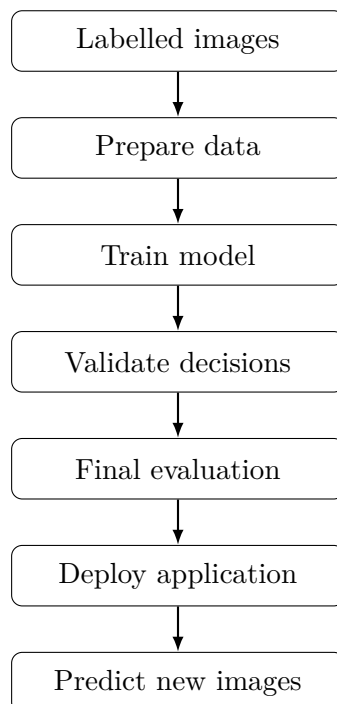
This is a *supervised-learning* problem because each training example is a labelled pair

$$(x_i, y_i),$$

where x_i is an image and y_i is its correct class.

2 Project Goals

The visible goal is to distinguish cats from dogs. The more important goal is to understand an entire machine-learning pipeline:



By the end of the project, we should understand:

- how images are represented numerically;
- how a convolutional neural network (CNN) extracts visual patterns;
- how loss, backpropagation, and gradient descent enable learning;
- why training, validation, and test sets have different roles;
- how overfitting occurs and how transfer learning helps;
- how to evaluate, interpret, and deploy a classifier.

3 Images as Numerical Data

The model receives a tensor of pixel values rather than seeing a cat or dog in the human sense. A colour image is commonly represented as

$$X \in \mathbb{R}^{C \times H \times W},$$

where $C = 3$ represents the red, green, and blue channels, while H and W denote image height and width. A 224×224 RGB image therefore contains

$$3 \times 224 \times 224 = 150,528$$

numerical values.

Typical preprocessing includes resizing or cropping, tensor conversion, pixel normalization, and—during training only—data augmentation. Training and inference must use compatible preprocessing rules.

4 Why Use a Convolutional Neural Network?

A CNN uses small learned filters that move across an image while preserving its spatial structure. Early layers often detect edges, lines, colour transitions, and simple textures. Deeper layers combine them into patterns such as fur, eyes, ears, noses, and body shapes:

$$\text{pixels} \rightarrow \text{edges} \rightarrow \text{textures} \rightarrow \text{parts} \rightarrow \text{class}.$$

A convolution can be written as

$$z_{i,j} = \sum_{a,b,c} K_{a,b,c} X_{i+a,j+b,c} + b,$$

where K is a learned kernel and b is a bias term. Training discovers useful kernel values instead of relying on manually programmed rules.

5 How the Model Learns

For every batch of training examples:

1. images pass through the network;
2. the network produces class scores called *logits*;
3. a loss function compares the predictions with the true labels;
4. backpropagation computes gradients;

5. an optimizer updates the parameters.

Abstractly, gradient descent performs the update

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L(\theta_t),$$

where θ denotes the model parameters, L the loss function, $\nabla_{\theta} L$ its gradient, and η the learning rate.

For two-class classification, cross-entropy loss can be written as

$$L = - \sum_{k=1}^2 y_k \log(p_k).$$

If the true class is dog, the loss is large when the model assigns dog a low probability. One *epoch* means that the model has processed the complete training dataset once.

6 Transfer Learning

Training a strong CNN from random parameters requires substantial data and computation. Instead, we can begin with a MobileNet or ResNet model pretrained on ImageNet. Such a model has already learned general visual features. We then replace its final classification layer with a new cat–dog head.

Initially, most pretrained layers are frozen and only the new head is trained. Later, selected deeper layers may be unfrozen and fine-tuned with a smaller learning rate:

general visual knowledge + cat/dog data $\longrightarrow$ specialized classifier.

A small CNN trained from scratch can serve as an educational baseline for comparison with transfer learning.

7 Dataset Separation

Subset	Purpose
Training	Updates model parameters
Validation	Selects settings and the best checkpoint
Test	Estimates final performance on unseen data

A possible split is 70% training, 15% validation, and 15% testing. The exact percentages are less important than preserving the three distinct roles. The test set must not influence architecture selection, augmentation, hyperparameters, confidence thresholds, or early stopping. Near-duplicate images must not appear in different subsets, because that would cause data leakage and overly optimistic results.

8 Generalization and Overfitting

The aim is not to memorize the training images but to generalize to unseen examples. A common sign of overfitting is that training loss continues to fall while validation performance stops improving or becomes worse.

Possible countermeasures include data augmentation, transfer learning, weight decay, early stopping, an appropriately sized model, and more varied data. Augmentation encodes assumptions such as: a horizontally flipped dog is still a dog.

9 Evaluation

For balanced classes, accuracy is a useful central metric:

$$\text{Accuracy} = \frac{\text{correct predictions}}{\text{all predictions}}.$$

We should also compute

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN},$$

and

$$F_1 = 2 \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

The final evaluation should include test accuracy, precision, recall, F_1 , a confusion matrix, training and validation curves, and examples of incorrect predictions. Error analysis can reveal problems caused by unusual angles, multiple animals, poor lighting, small subjects, confusing backgrounds, and dataset bias.

10 Confidence Is Not Certainty

Softmax converts logits z_i into scores that sum to one:

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}.$$

However, a score such as “dog: 97%” does not guarantee that the model is correct 97% of the time in comparable situations. Neural networks can be overconfident.

Furthermore, the model operates in the closed world

$$\{\text{cat}, \text{dog}\}.$$

If it receives an image of a car, it must still select one of these classes. A low-confidence warning is useful but does not solve out-of-distribution detection. The application must communicate this limitation clearly.

11 Project Scope

The first version should:

- train a reproducible binary classifier;
- compare a simple CNN with transfer learning;
- save the best model checkpoint;
- evaluate it on an untouched test set;
- analyse errors;
- accept uploaded images and display predictions;
- warn users about the model’s limitations.

It should not initially attempt animal detection in crowded images, breed classification, video processing, reliable “neither” recognition, a large custom architecture, or mobile deployment. Grad-CAM is a valuable optional extension because it can reveal whether the classifier focuses on the animal or on irrelevant background patterns.

12 Research Questions

The final project should answer:

1. How well does a small CNN trained from scratch perform?
2. How much does transfer learning improve performance?
3. Does augmentation improve validation performance?
4. Which kinds of images cause failures?
5. Does the model focus on the animal or the background?
6. How reliable are its confidence scores?
7. What happens with images containing neither a cat nor a dog?

13 Final Objective

Design and evaluate a reproducible computer-vision pipeline that classifies unseen images as cats or dogs, comparing a simple CNN with transfer learning and investigating generalization, confidence, failure cases, and practical deployment.

The next step is to study the dataset: choose a source, inspect its structure, identify duplicates or corrupted images, and design fair training, validation, and test subsets before implementing the model.